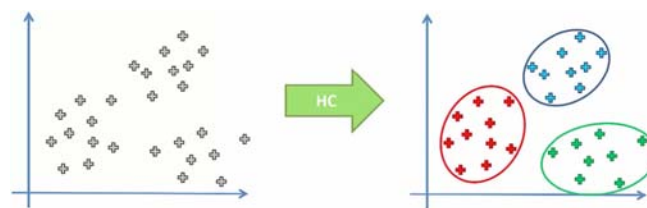


Hierarchical Clustering

● Regroupement hiérarchique (Hierarchical Clustering : HC)

● Définition

- Le regroupement hiérarchique est une technique populaire d'apprentissage automatique non supervisé utilisée pour regrouper des points de données similaires en grappes (clusters) sur la base de leur similarité.
- Cette technique est dite "hiérarchique" parce qu'elle crée une hiérarchie arborescente de grappes, où chaque nœud représente une grappe qui peut être divisée en sous-grappes plus petites.
- Il existe deux types de techniques de regroupement hiérarchique :
 - les techniques agglomératives (**Agglomerative Clustering**)
 - les techniques divisives (**Divisive Clustering**)



106

Hierarchical Clustering

● Regroupement hiérarchique (Hierarchical Clustering : HC)

● Regroupement agglomératif (Agglomerative Clustering)

- Le regroupement agglomératif est un type d'algorithme de regroupement hiérarchique qui fusionne les paires de points de données ou de grappes les plus similaires, en construisant une hiérarchie de grappes jusqu'à ce que tous les points de données appartiennent à une seule grappe.
- Chaque point de données est d'abord considéré comme son propre cluster, puis les paires de clusters les plus similaires sont fusionnées de manière itérative jusqu'à ce que tous les points de données appartiennent à un seul et même cluster.

● Regroupement par division (Divisive Clustering)

- Le clustering divisé est une technique qui commence avec tous les points de données dans un seul cluster et divise récursivement les clusters en sous-groupes plus petits basés sur leur dissimilarité.
- Elle est également connue sous le nom de regroupement "descendant".

107

Hierarchical Clustering

● Regroupement hiérarchique agglomératif (Agglomerative Clustering)

● Algorithme

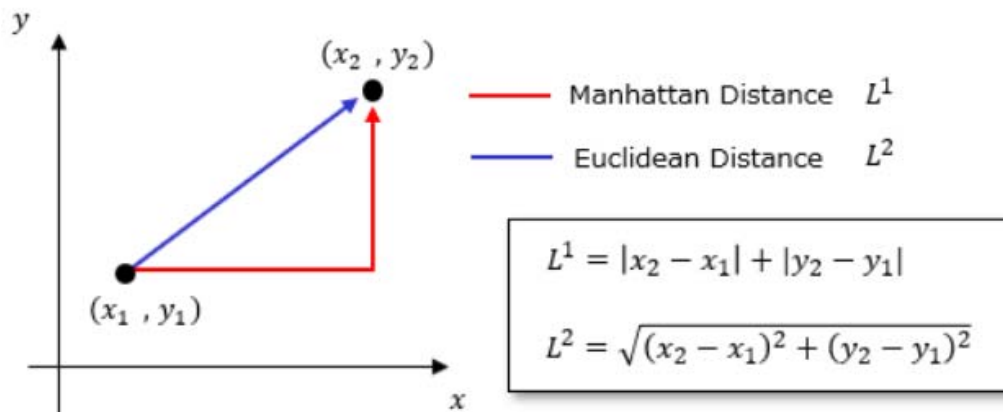
- Étape 1 : faire de chaque point de données un cluster à point unique
↓ → Former N clusters
- Étape 2 : prendre les deux points de données les plus proches et en faire un seul cluster
↓ → Former N-1 clusters
- Étape 3 : prendre les deux clusters les plus proches et en faire un seul cluster
↓ → Former N-2 clusters
- Étape 4 : répéter l'étape 3 jusqu'à ce qu'il n'y ait plus qu'un seul cluster
↓
- Fin

108

Hierarchical Clustering

● Regroupement hiérarchique agglomératif (Agglomerative Clustering)

● Calcul de distance

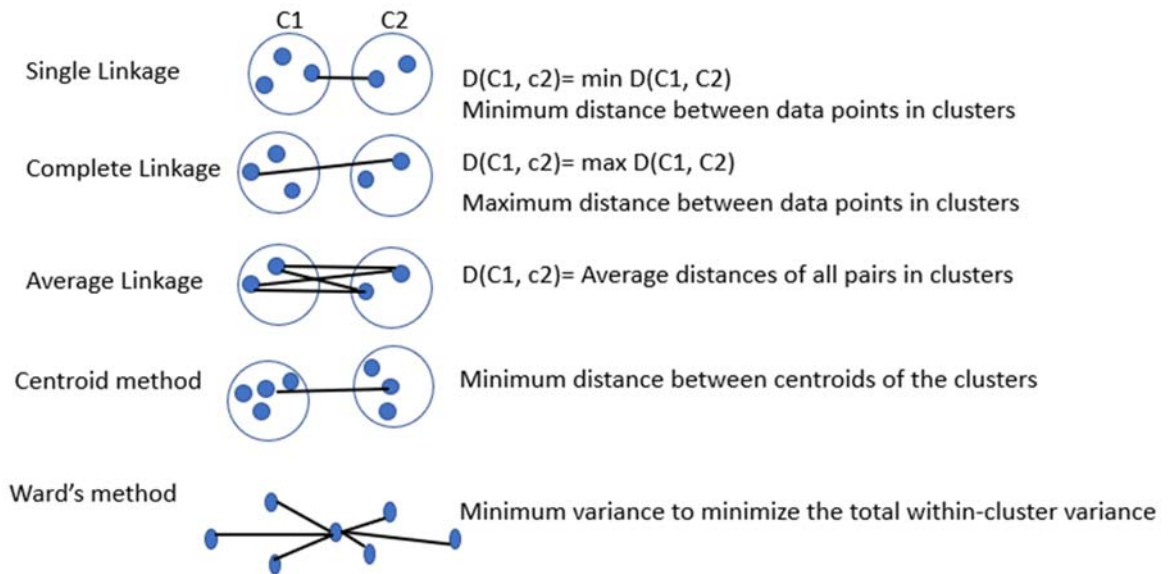


109

Hierarchical Clustering

● Regroupement hiérarchique agglomératif (Agglomerative Clustering)

● Méthodes de liaison

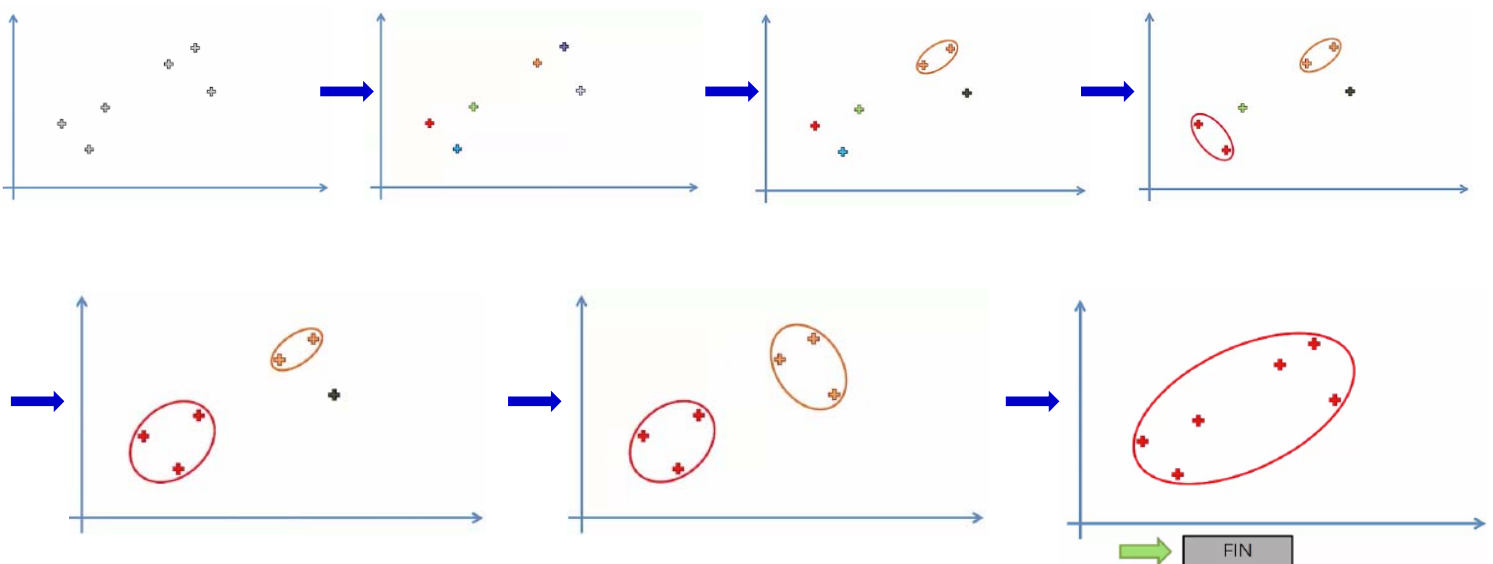


3

Hierarchical Clustering

● Regroupement hiérarchique agglomératif (Agglomerative Clustering)

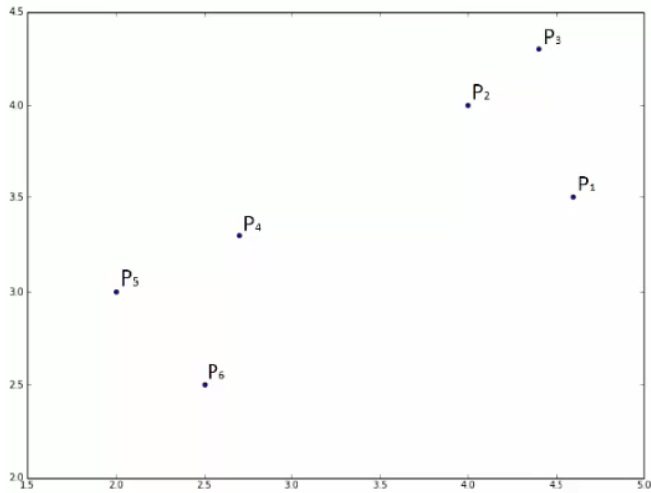
● Exemple



Hierarchical Clustering

● Regroupement hiérarchique agglomératif (Agglomerative Clustering)

● Fonctionnement des dendrogrammes

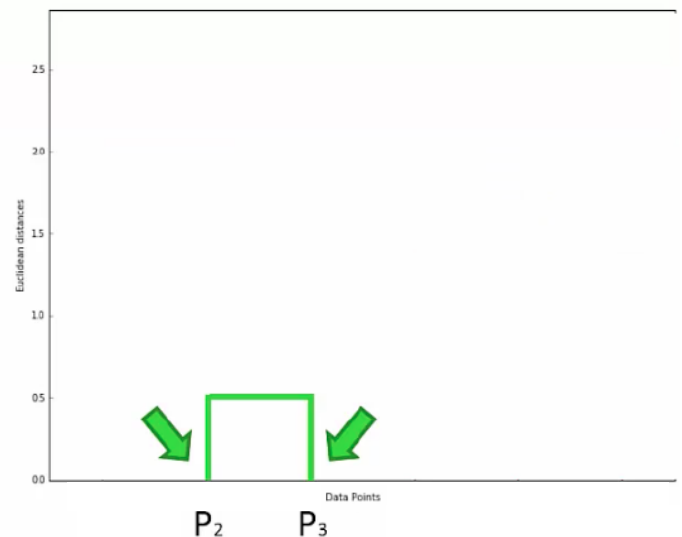
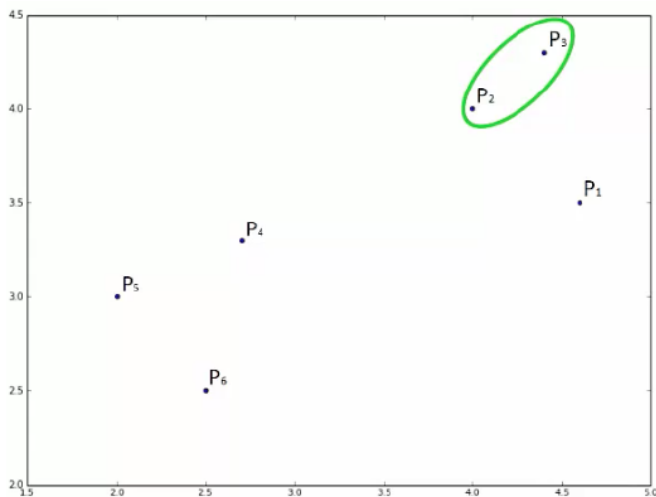


112

Hierarchical Clustering

● Regroupement hiérarchique agglomératif (Agglomerative Clustering)

● Fonctionnement des dendrogrammes

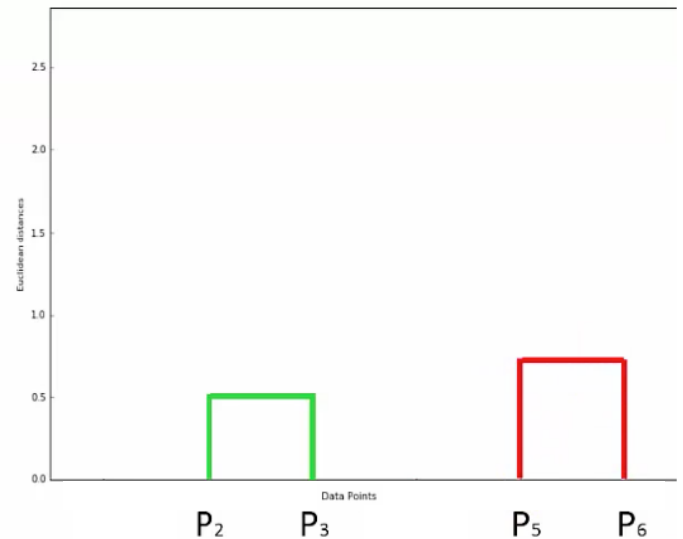
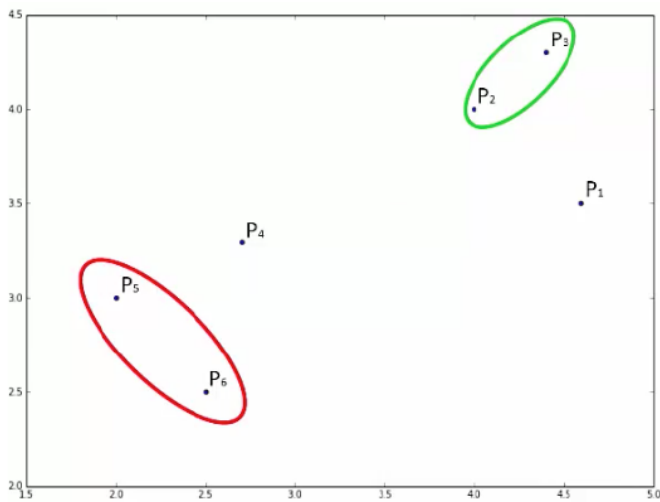


113

Hierarchical Clustering

● Regroupement hiérarchique agglomératif (Agglomerative Clustering)

● Fonctionnement des dendrogrammes

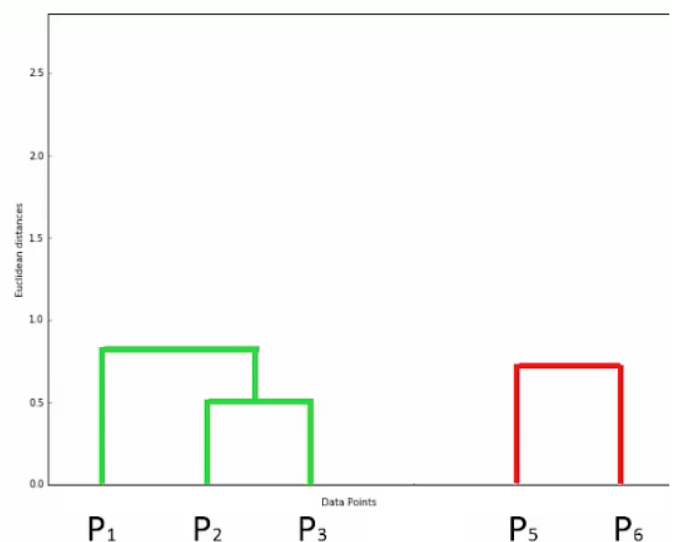
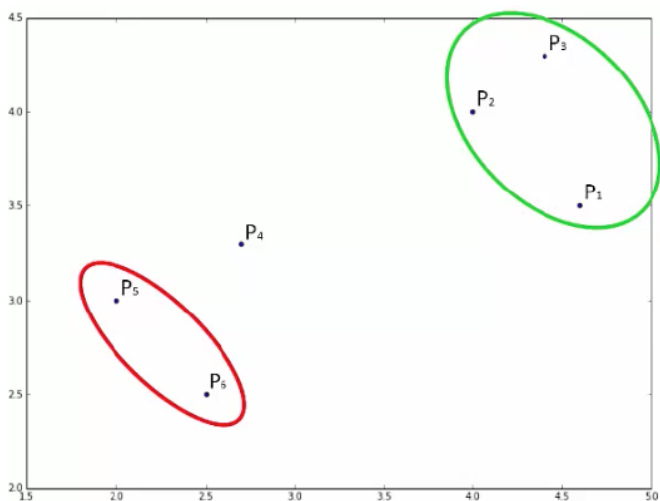


114

Hierarchical Clustering

● Regroupement hiérarchique agglomératif (Agglomerative Clustering)

● Fonctionnement des dendrogrammes

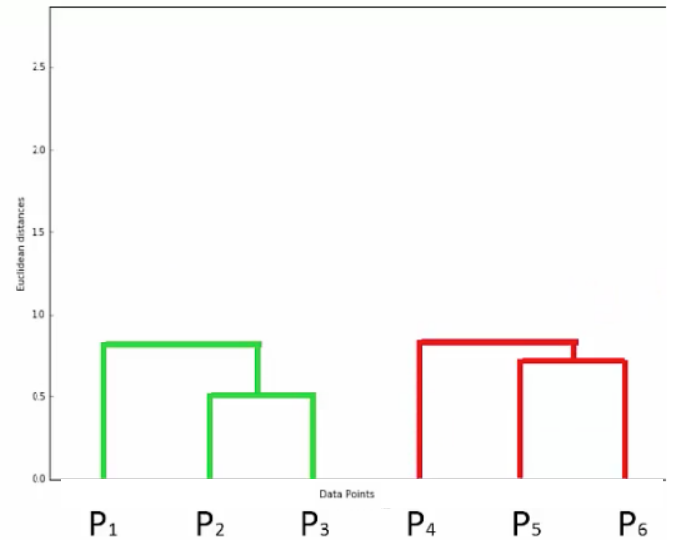
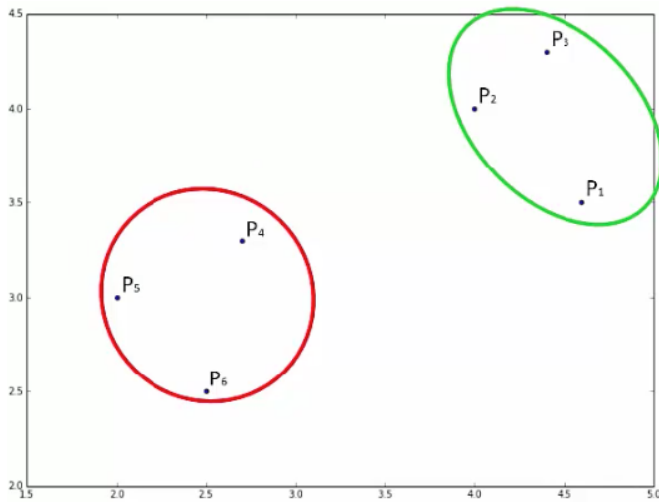


115

Hierarchical Clustering

● Regroupement hiérarchique agglomératif (Agglomerative Clustering)

● Fonctionnement des dendrogrammes

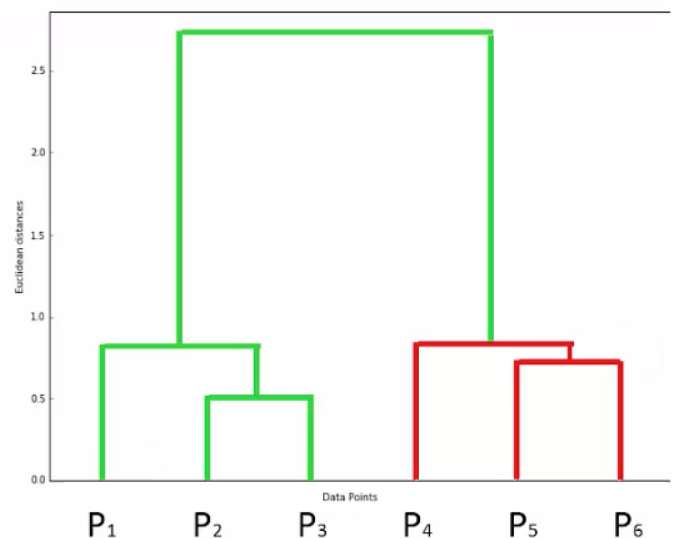
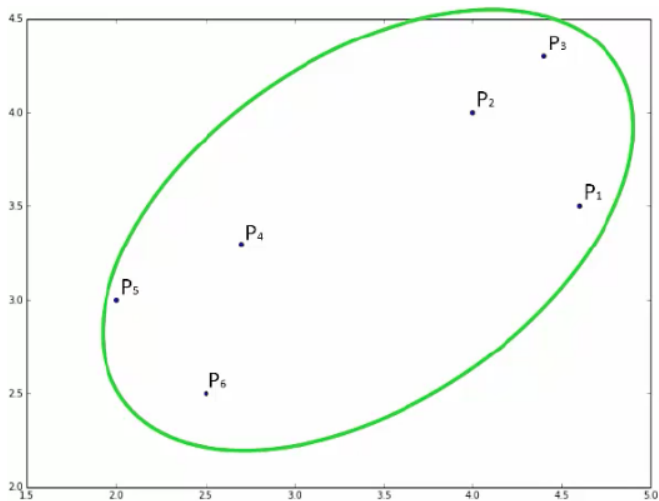


116

Hierarchical Clustering

● Regroupement hiérarchique agglomératif (Agglomerative Clustering)

● Fonctionnement des dendrogrammes



117

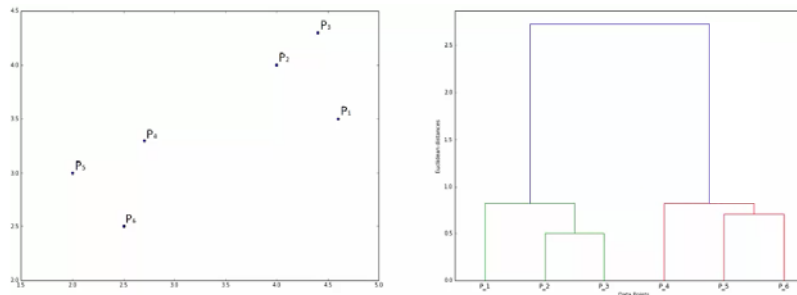
Hierarchical Clustering

● Regroupement hiérarchique agglomératif (Agglomerative Clustering)

● Recherche du nombre optimal de clusters

- Faire déplacer une droite horizontale le long du dendrogramme verticalement (de bas en haut)
- Le nombre de points d'intersection de cette droite avec le dendrogramme représente le nombre de clusters
- Chercher la distance la plus grande avec un nombre de points d'intersection fixe
 - Ce nombre de points d'intersection correspond au nombre optimal de clusters

● Exemple

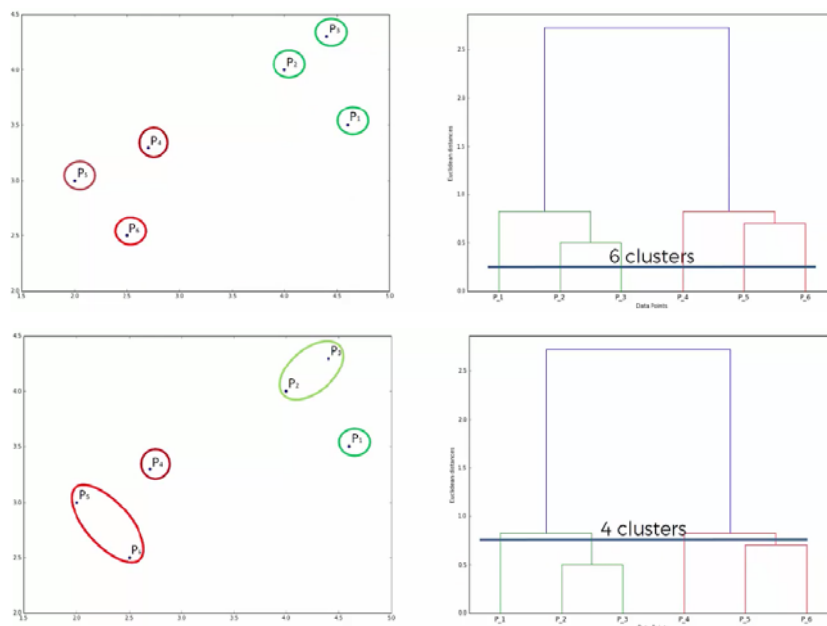


118

Hierarchical Clustering

● Regroupement hiérarchique agglomératif (Agglomerative Clustering)

● Exemple

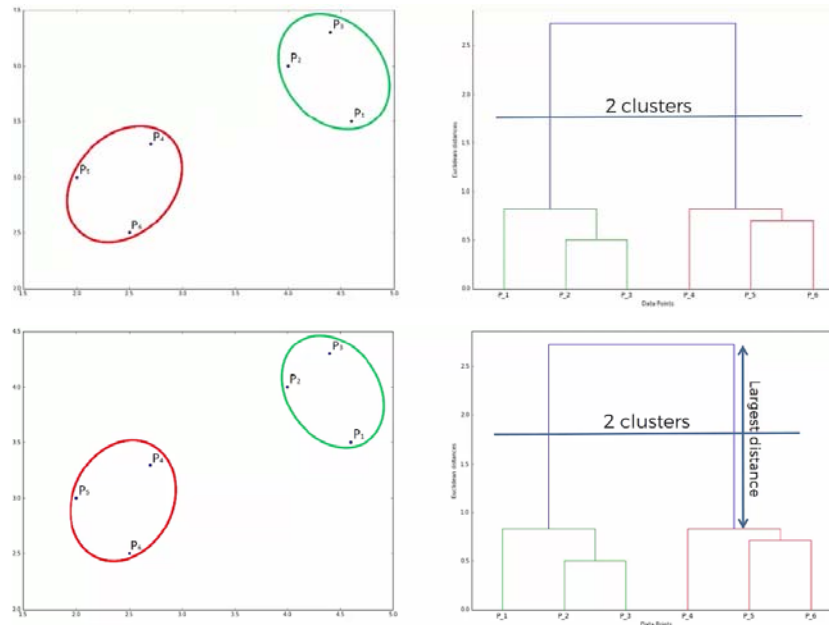


119

Hierarchical Clustering

● Regroupement hiérarchique agglomératif (Agglomerative Clustering)

● Exemple



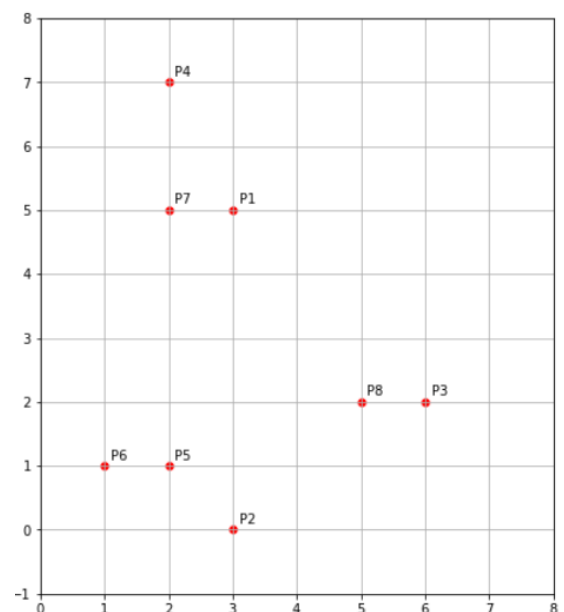
120

Hierarchical Clustering

● Regroupement hiérarchique agglomératif (Agglomerative Clustering)

● Exercice

- En se basant sur la distance de manhattan et la liaison de type single, tracer le dendrogramme relatif à l'ensemble de ces données : spécifier le tableau des distances à chaque regroupement.
- Déduire le nombre de clusters optimal.
- Effectuer le regroupement hiérarchique.

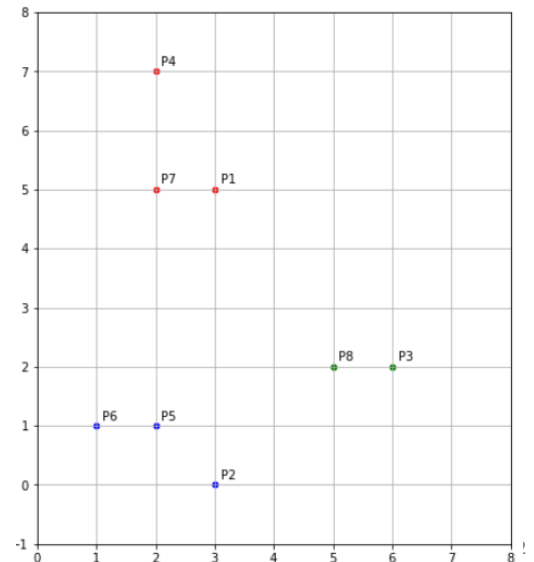
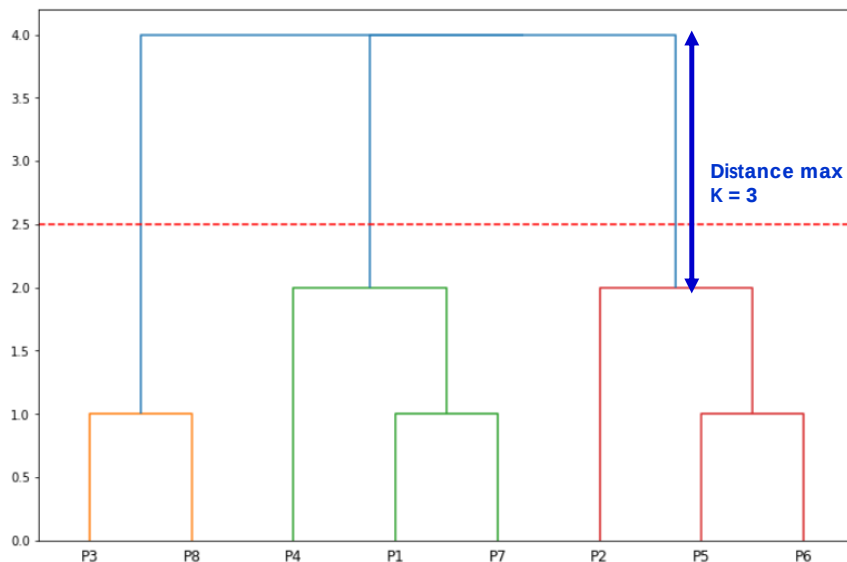


121

Hierarchical Clustering

● Regroupement hiérarchique agglomératif (Agglomerative Clustering)

● Solution



Hierarchical Clustering

● Regroupement hiérarchique agglomératif (Agglomerative Clustering)

● Atelier

'synthetic_data.csv'

	var1	var2
0	-6.11	1.47
1	-7.50	0.91
2	-10.84	-7.55
3	-4.54	-2.14
4	-1.91	-0.03
...	...	...
1495	-8.53	-6.73
1496	-3.27	2.24
1497	1.65	-0.20
1498	0.79	0.61
1499	1.91	0.25

Hierarchical Clustering

Atelier

- Importation des packages

```
import pandas as pd
import matplotlib.pyplot as plt
```

- Lecture

```
X = pd.read_csv('synthetic_data.csv')
```

- Informations

```
X.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1500 entries, 0 to 1499
Data columns (total 2 columns):
#   Column  Non-Null Count  Dtype  
---  -
0   var1     1500 non-null    float64
1   var2     1500 non-null    float64
```

	var1	var2
0	-6.11	1.47
1	-7.50	0.91
2	-10.84	-7.55
3	-4.54	-2.14
4	-1.91	-0.03
...	...	...
1495	-8.53	-6.73
1496	-3.27	2.24
1497	1.65	-0.20
1498	0.79	0.61
1499	1.91	0.25

124

Hierarchical Clustering

Atelier

- Recherche des outliers

```
import seaborn as sns
sns.boxplot(data=X, orient='v')
```

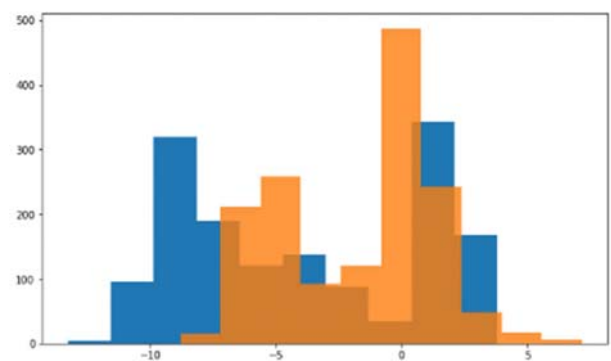
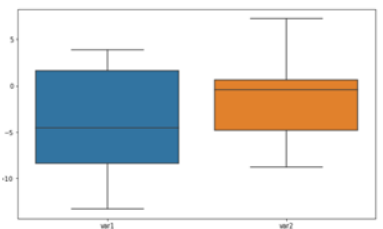
- Données manquantes

```
X.isnull().sum()

var1    0
var2    0
```

- Mise à l'échelle

```
plt.hist(X['var1'])
plt.hist(X['var2'], alpha=0.8)
plt.show()
```



125

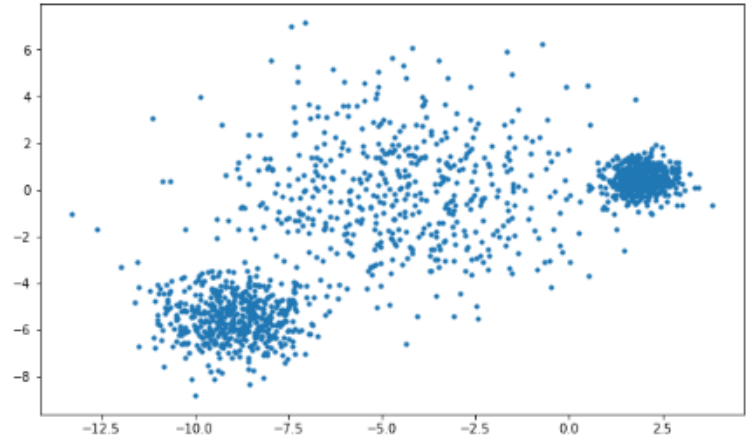
Hierarchical Clustering

Atelier

Visualisation des données

```
X = X.values
```

```
plt.rcParams["figure.figsize"] = (10,6)
plt.scatter(X[:,0], X[:,1], s=10)
plt.show()
```



126

Hierarchical Clustering

Atelier

Calcul des distances

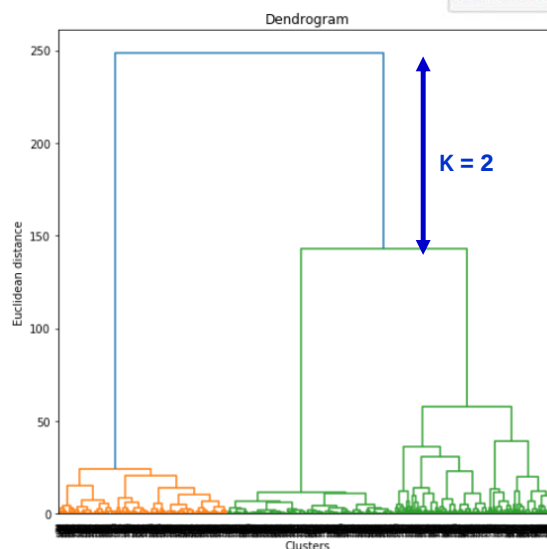
```
import scipy.cluster.hierarchy as sch
Z = sch.linkage(X, method="ward")
```

```
np.set_printoptions(suppress = True)
Z
```

```
[[1003. , 1256. ,  0. ,  2. ],
 [ 595. , 1304. ,  0. ,  2. ],
 [ 961. , 1097. ,  0. ,  2. ],
 ...,
 [2994. , 2995. , 58.07, 485. ],
 [2981. , 2996. , 142.86, 990. ],
 [2992. , 2997. , 248.48, 1500. ]]
```

Affichage du dendrogramme

```
den = sch.dendrogram(Z)
plt.title('Dendrogram')
plt.xlabel('Clusters')
plt.ylabel('Euclidean distance')
plt.show()
```



127

Hierarchical Clustering

Atelier

Regroupement

```
from sklearn.cluster import AgglomerativeClustering
model_ac = AgglomerativeClustering(n_clusters = 2, metric='euclidean', linkage='ward')
model_ac.fit(X)
```

Visualisation

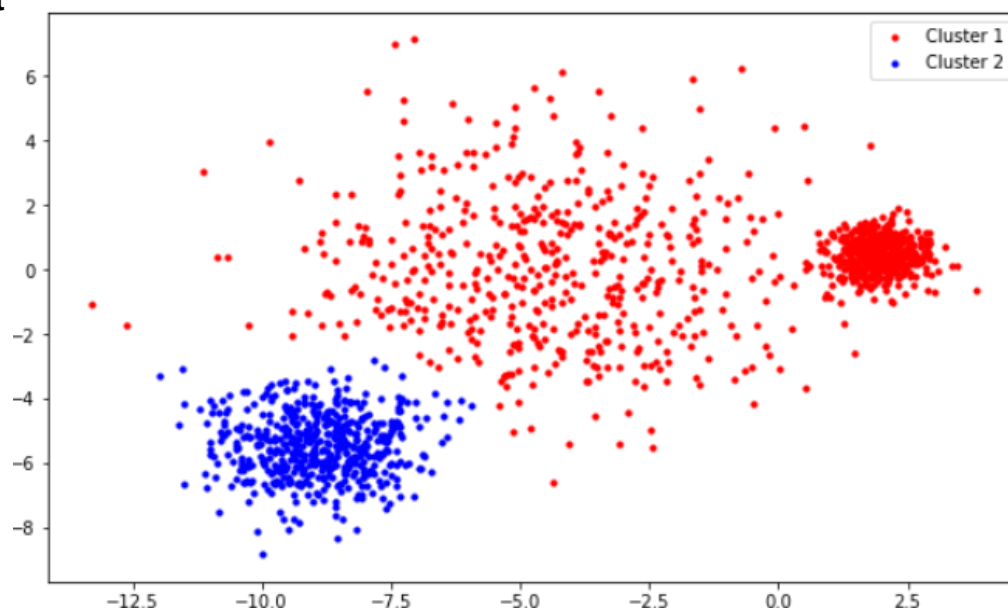
```
colors=['red','blue','green']
for i in range(0,2):
    plt.scatter(X[model_ac.labels_ == i, 0], X[model_ac.labels_ == i, 1],
                s = 10, c=colors[i], label = 'Cluster {}'.format(i+1))
plt.legend()
plt.show()
```

128

Hierarchical Clustering

Atelier

Résultat



129

Hierarchical Clustering

Atelier

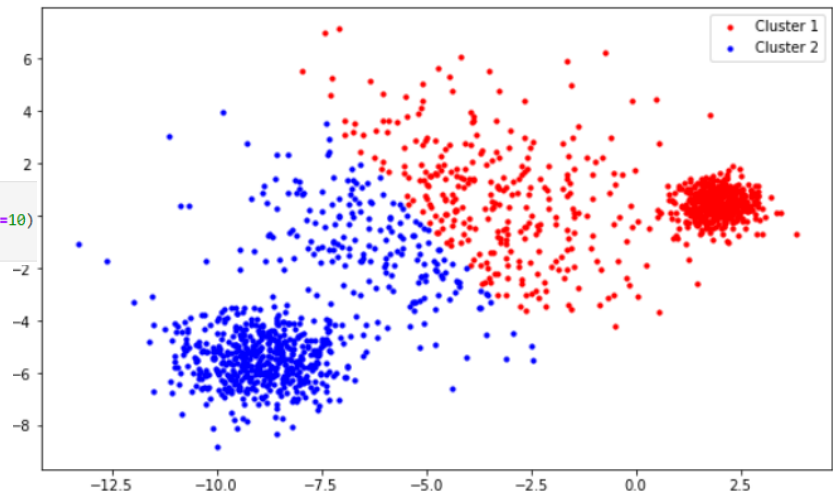
Comparaison par rapport à K-means

Modèle

```
from sklearn.cluster import KMeans
model_km = KMeans(n_clusters = 2, init = 'k-means++', random_state = 0, n_init=10)
model_km.fit(X)
```

Affichage

```
for i in range(0,2):
    plt.scatter(X[model_km.labels_ == i, 0], X[model_km.labels_ == i, 1],
                s = 10, c=colors[i], label = 'Cluster {}'.format(i+1))
plt.legend()
plt.show()
```



130

Hierarchical Clustering

Atelier

Test avec k=3

Regroupement

```
model_ac = AgglomerativeClustering(n_clusters = 3, metric='euclidean', linkage='ward')
model_ac.fit(X)
```

Visualisation

```
for i in range(0,3):
    plt.scatter(X[model_ac.labels_ == i, 0], X[model_ac.labels_ == i, 1],
                s = 10, c=colors[i], label = 'Cluster {}'.format(i+1))
plt.legend()
plt.show()
```

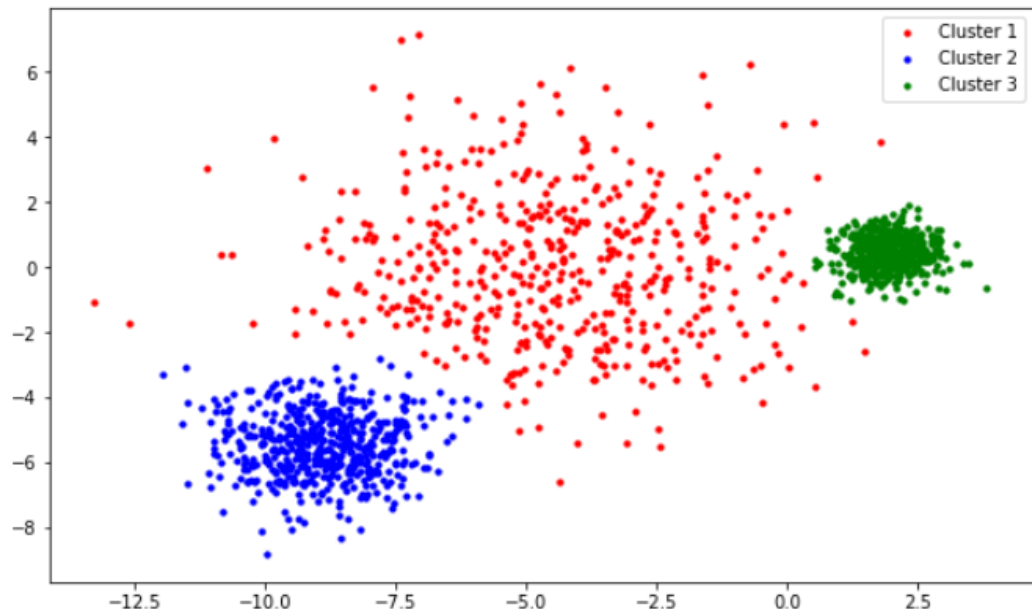
131

Hierarchical Clustering

Atelier

Test avec k=3

Résultat



132

Hierarchical Clustering

Atelier

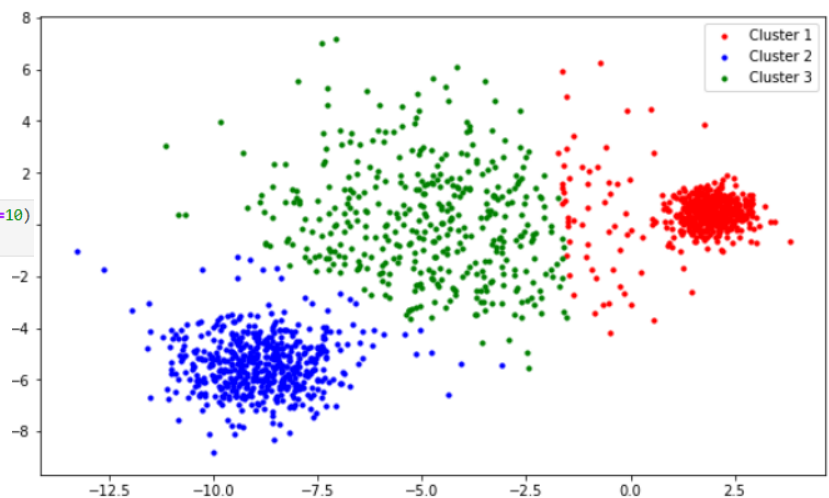
Comparaison par rapport à K-means

Modèle

```
model_km = KMeans(n_clusters = 3, init = 'k-means++', random_state = 0, n_init=10)
model_km.fit(X)
```

Affichage

```
for i in range(0,3):
    plt.scatter(X[model_km.labels_ == i, 0], X[model_km.labels_ == i, 1],
                s = 10, c=colors[i], label = 'Cluster {}'.format(i+1))
plt.legend()
plt.show()
```

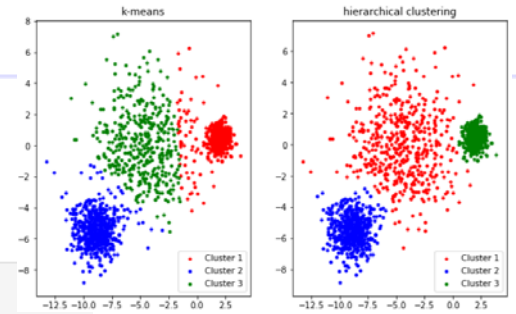


133

Hierarchical Clustering

Atelier

Récapitulatif



```
plt.subplot(1,2,1)
for i in range(0,3):
    plt.scatter(X[model_km.labels_ == i, 0], X[model_km.labels_ == i, 1],
                s = 10, c=colors[i], label = 'Cluster {}'.format(i+1))
plt.title('k-means')
plt.legend()

plt.subplot(1,2,2)
for i in range(0,3):
    plt.scatter(X[model_ac.labels_ == i, 0], X[model_ac.labels_ == i, 1],
                s = 10, c=colors[i], label = 'Cluster {}'.format(i+1))
plt.title('hierarchical clustering')
plt.legend()

plt.show()
```

134

Hierarchical Clustering

Atelier

Étude des méthodes de liaison

```
single = AgglomerativeClustering(n_clusters = 3, metric='euclidean', linkage='single')
single.fit(X)
```

► AgglomerativeClustering ⓘ ⓘ

```
average = AgglomerativeClustering(n_clusters = 3, metric='euclidean', linkage='average')
average.fit(X)
```

► AgglomerativeClustering ⓘ ⓘ

```
complete = AgglomerativeClustering(n_clusters = 3, metric='euclidean', linkage='complete')
complete.fit(X)
```

► AgglomerativeClustering ⓘ ⓘ

```
ward = AgglomerativeClustering(n_clusters = 3, metric='euclidean', linkage='ward')
ward.fit(X)
```

► AgglomerativeClustering ⓘ ⓘ

135

Hierarchical Clustering

Atelier

Étude des méthodes de liaison

```
▶ plt.rcParams["figure.figsize"] = (12,10)

plt.subplot(2,2,1)
for i in range(0,3):
    plt.scatter(X[single.labels_ == i, 0], X[single.labels_ == i, 1],
                s = 10, c=colors[i], label = 'Cluster {}'.format(i+1))
plt.title('single : score=%.2f'%silhouette_score(X,single.labels_))
plt.legend()

plt.subplot(2,2,2)
for i in range(0,3):
    plt.scatter(X[complete.labels_ == i, 0], X[complete.labels_ == i, 1],
                s = 10, c=colors[i], label = 'Cluster {}'.format(i+1))
plt.title('complete : score=%.2f'%silhouette_score(X,complete.labels_))
plt.legend()
```

136

Hierarchical Clustering

Atelier

Étude des méthodes de liaison

```
plt.subplot(2,2,3)
for i in range(0,3):
    plt.scatter(X[average.labels_ == i, 0], X[average.labels_ == i, 1],
                s = 10, c=colors[i], label = 'Cluster {}'.format(i+1))
plt.title('average : score=%.2f'%silhouette_score(X,average.labels_))
plt.legend()

plt.subplot(2,2,4)
for i in range(0,3):
    plt.scatter(X[ward.labels_ == i, 0], X[ward.labels_ == i, 1],
                s = 10, c=colors[i], label = 'Cluster {}'.format(i+1))
plt.title('ward : score=%.2f'%silhouette_score(X,ward.labels_))
plt.legend()

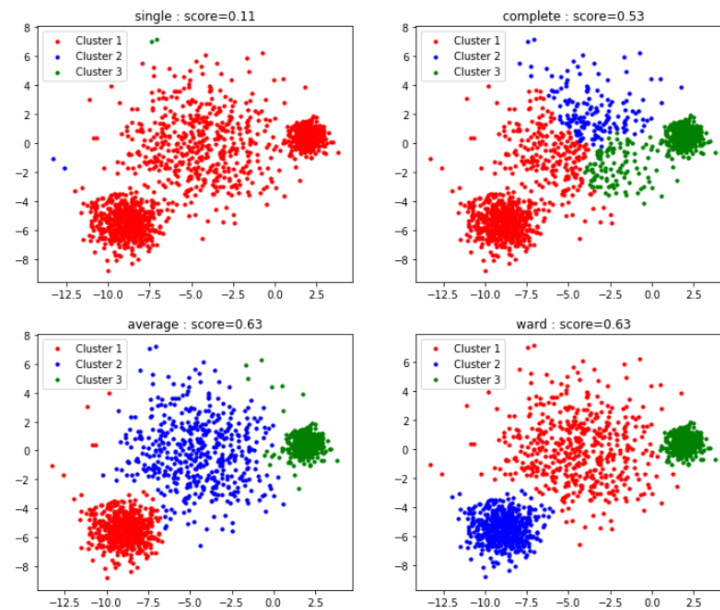
plt.show()
```

137

Hierarchical Clustering

Atelier

Étude des méthodes de liaison



138

Hierarchical Clustering

Atelier

Étude des méthodes de liaison

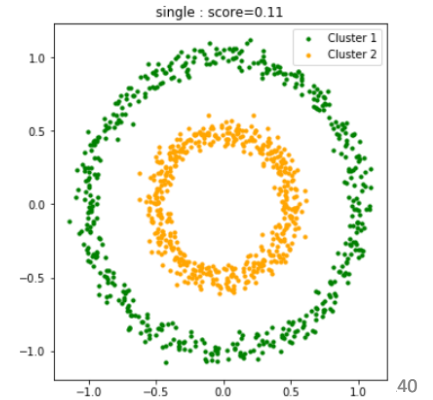
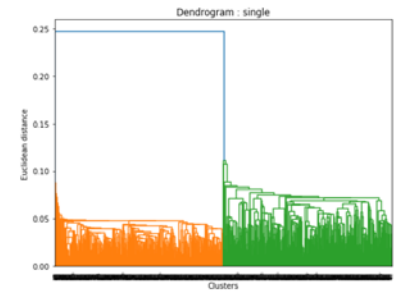
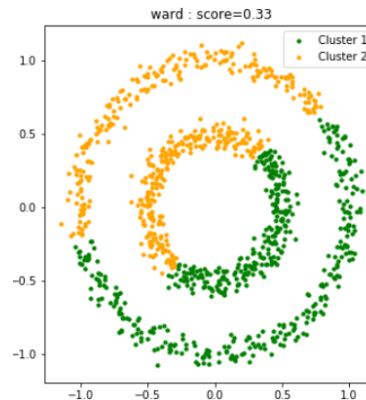
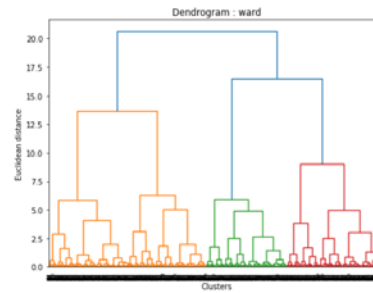
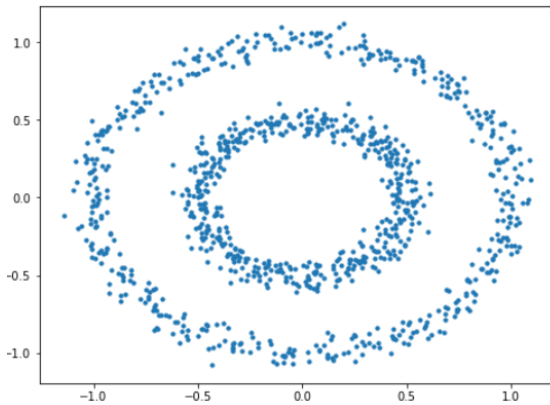
- Single :
 - Il est rapide et peut donner de bons résultats sur des données non globulaires, mais ses performances sont médiocres en présence de bruit.
- Average, Complete :
 - Ils donnent de bons résultats sur des grappes globulaires proprement séparées, mais ont des résultats mitigés dans les autres cas.
- Ward
 - C'est la méthode la plus efficace pour les données bruitées.

139

Hierarchical Clustering

Atelier

- Étude des méthodes de liaison
- Autre exemple



Hierarchical Clustering

Avantages

- Facile à implémenter
- Fournit une représentation visuelle des données : il produit un dendrogramme montrant les relations entre les différents groupes ce qui facilite la compréhension et l'interprétation des données.
- Résistant aux valeurs aberrantes

Inconvénients

- Coûteux en termes de calcul, en particulier pour les grands ensembles de données.
- Peu sensible au bruit, ce qui peut affecter la qualité des résultats.
- En raison de sa complexité informatique, il n'est pas adapté aux grands ensembles de données.